V1T — Capture The Flag Event Write-ups

OSINT, Miscellaneous & Reverse Engineering

Contents

1 Challenge 1 — Hack The Box **2**

2 Challenge 2 — J2 Team **2**

3 Challenge 3 — Rules **3**

4 Challenge 4 — Discord **5**

5 Challenge 5 — Town Tour 01 **5**

6 Challenge 6 — Sun Set **8**

7 Challenge 7 — Classless **9**

8 Challenge 8 — Atari 2600 **10**

9 Challenge 9 — Town Tour 03 **13**

Challenge 1 — Hack The Box

Category

Miscellaneous / Sponsor Challenge

Problem Description

This is an introductory challenge created to acknowledge Hack The Box as a sponsor of the V1T CTF event.

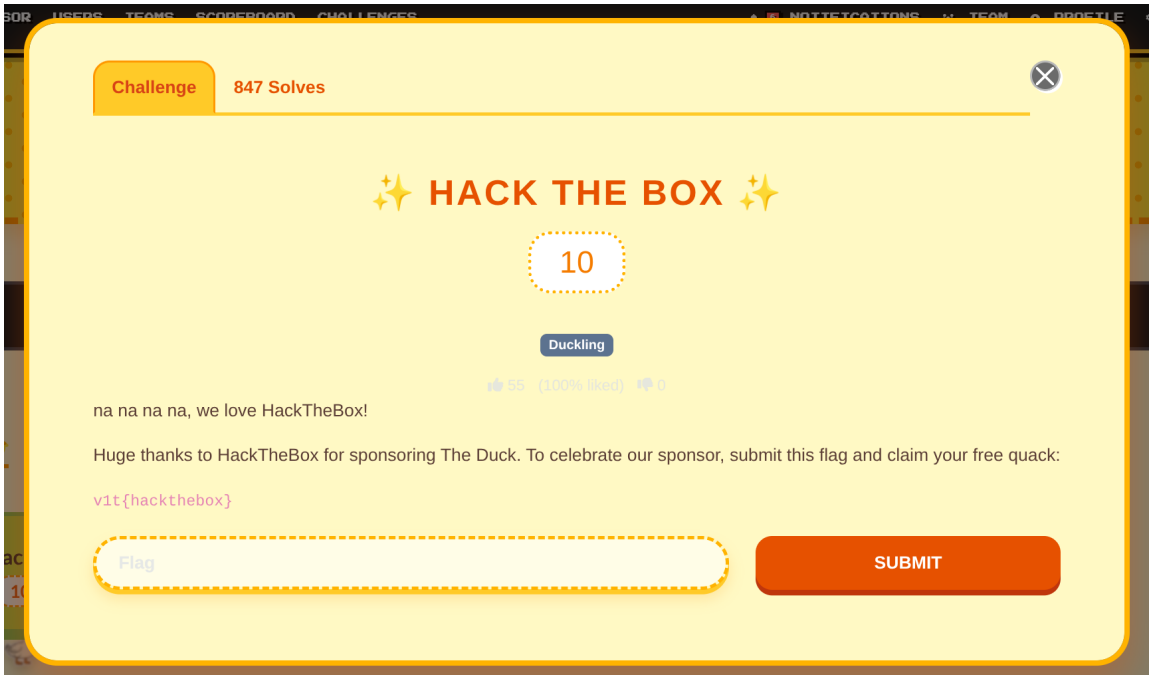


Figure 1: Challenge description for Hack The Box.

Approach and Solution

The flag was explicitly stated within the challenge description itself. No additional tools or steps were required.

Flag: `v1t{hackthebox}`

Challenge 2 — J2 Team

Category

Miscellaneous / Sponsor Challenge

Problem Description

This is an introductory challenge created to acknowledge J2 Team as a sponsor of the V1T CTF event.



Figure 2: Challenge description for J2 Team.

Approach and Solution

The flag was explicitly stated within the challenge description itself. No additional tools or steps were required.

Flag: `v1t{j2team}`

Challenge 3 — Rules

Category

Miscellaneous / Sanity Check

Problem Description

This challenge serves as a sanity check to verify that participants have read the event rules prior to competing.

Approach and Solution

Step 1. Locate the rules section: The challenge prompted participants to navigate to the rules section of the competition platform.

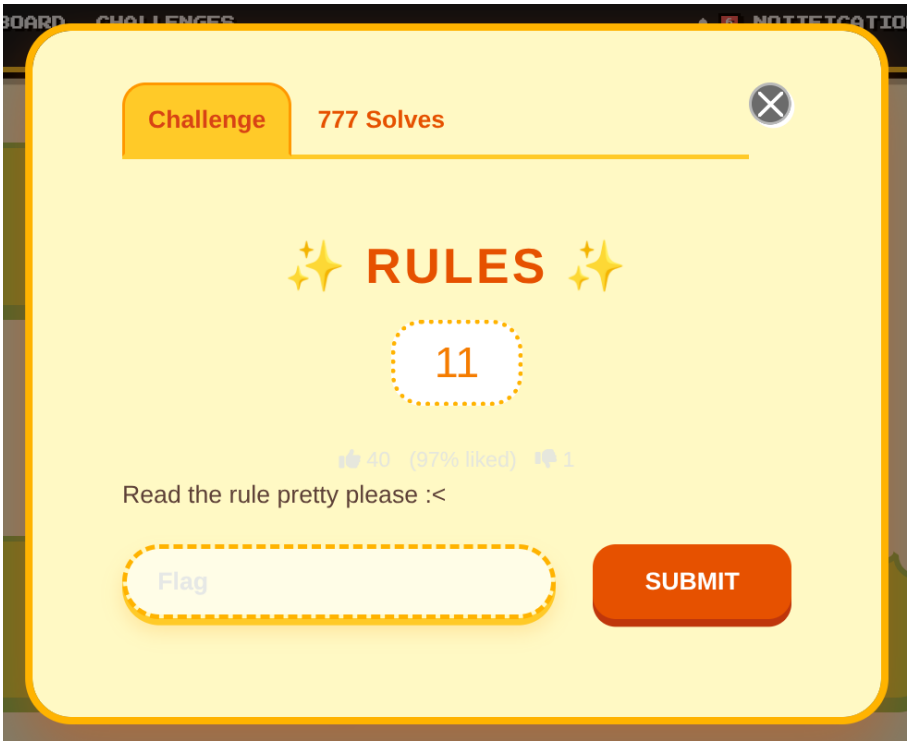


Figure 3: Challenge description for Rules.

Step 2. Retrieve the flag: The flag was found embedded within the rules section of the challenge platform.

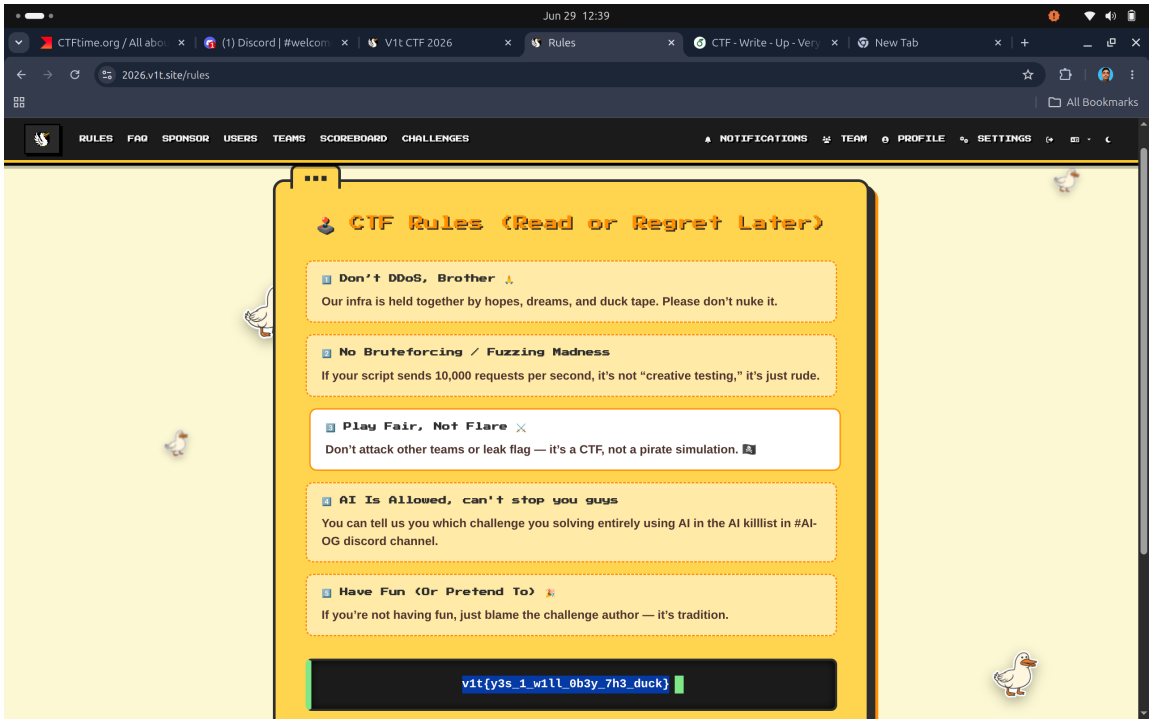


Figure 4: The flag visible within the rules section.

Flag: `v1t{y3s_1_w1ll_0b3y_7h3_duck}`

Challenge 4 — Discord

Category

Miscellaneous / Sanity Check

Problem Description

This challenge serves as a sanity check to verify that participants have joined and explored the V1T CTF Discord server.

Approach and Solution

Step 1. Join the Discord server: Participants were required to join the official V1T CTF Discord server.

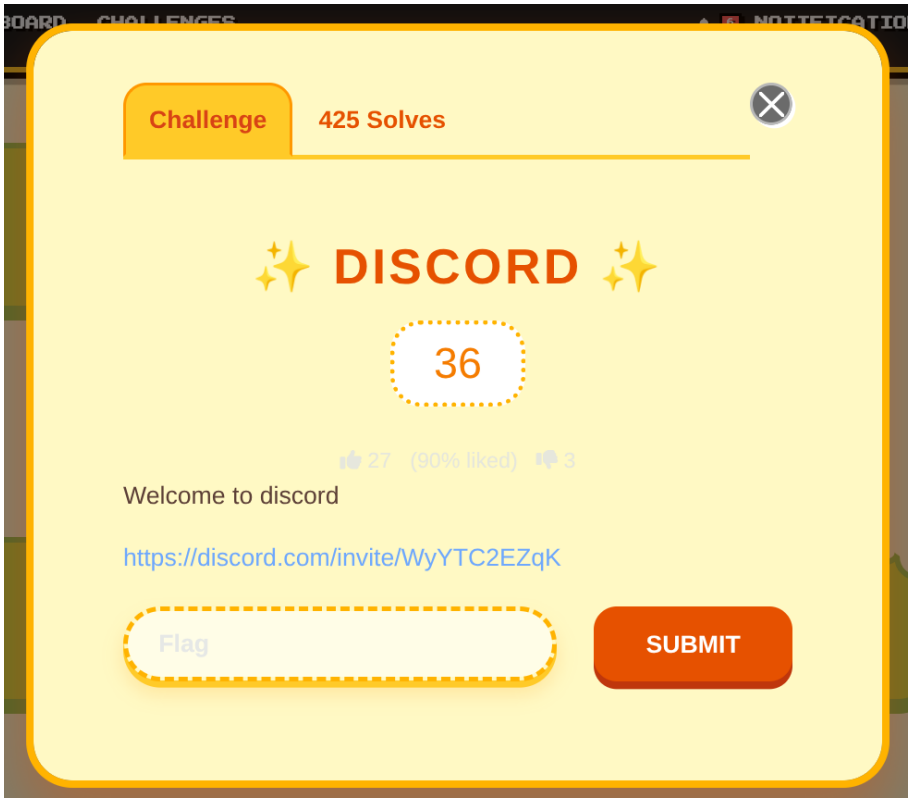


Figure 5: Challenge description for Discord.

Step 2. Locate the announcement channel: The flag was posted in the announcements section of the Discord server.

Flag: `v1t{duckc0rd}`

Challenge 5 — Town Tour 01

Category

OSINT (Open Source Intelligence)

Problem Description

The challenge states that there is a location at the author's friend's hometown where an exhibition of aircraft is held. Participants are required to identify the specific helicopter on display, whose designation forms the flag.



Figure 6: Challenge description for Town Tour 01.

Approach and Solution

Step 1. **Identify the location:** With the assistance of an LLM, the described location was identified as the **Hanoi Flag Tower** in Hanoi, Vietnam, which is situated adjacent to the Vietnam Military History Museum. The museum grounds feature an outdoor display of aircraft and military equipment.

Step 2. **Locate the helicopter via Google Maps:** Google Maps was used to navigate to the site. Using the 3D view, a helicopter was identified on the museum grounds.



Figure 7: Google Maps 3D view showing the helicopter on display at the museum grounds.

Step 3. Identify the helicopter model: With the help of an LLM, the helicopter was cross-referenced against the museum's known exhibits. The aircraft was identified as a **Mil Mi-8 Hip**, serial number 7822.

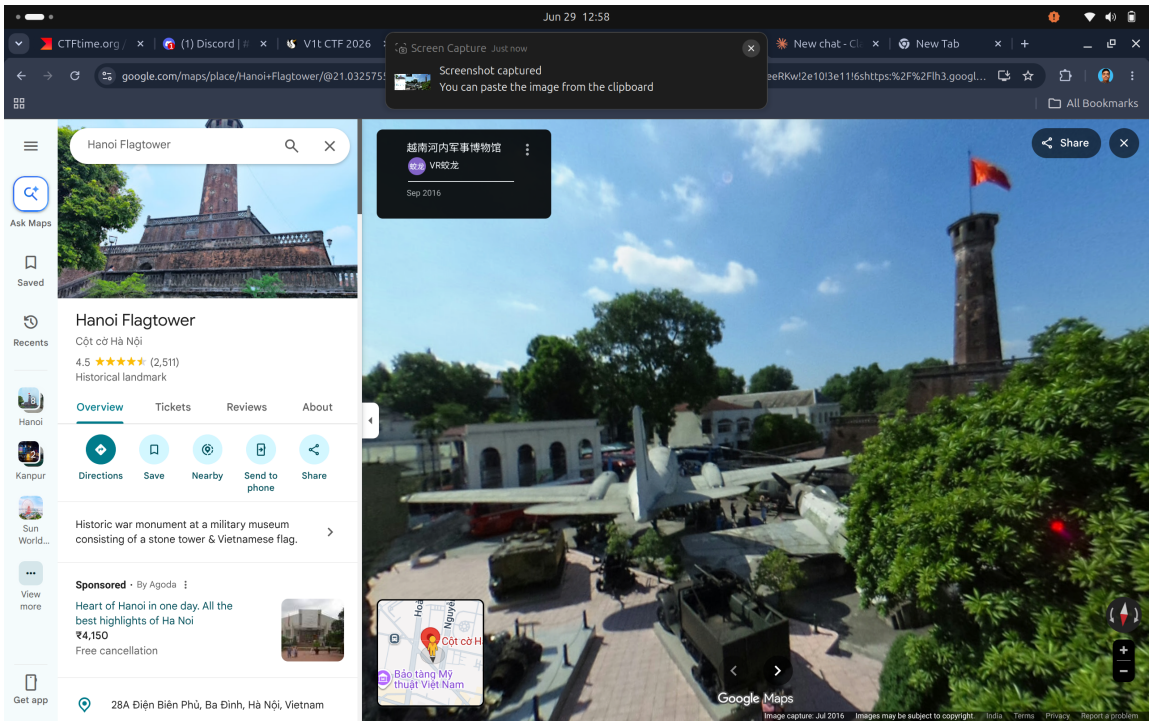


Figure 8: Identification of the helicopter model and serial number.

Flag: `V1T{Mil_Mi_8_Hip_7822}`

Challenge 6 — Sun Set

Category

OSINT (Open Source Intelligence)

Problem Description

The challenge describes a location where the sun appears closer than it actually is, featuring a calm lake with no ducks. Participants are required to identify the specific location to obtain the flag.

Approach and Solution

Step 1. **Analyse the description:** Key descriptors from the challenge were noted: a location where the sun appears unusually close, a calm lake, and an absence of ducks.

Step 2. **Identify the location:** With the assistance of an LLM, the location was identified as the area surrounding **Ho Bach Hop Thuy lake** within the Duong Noi Urban Area in Ha Dong District, Hanoi, Vietnam. The challenge description's reference to the sun appearing closer than it is pointed to a nearby astronomy park: **Cong vien Thien van hoc** (Astronomical Park).



Figure 9: Visual evidence and location identification for the Sun Set challenge.

Flag: `v1t{cong_vien_thien_van_hoc}`

Challenge 7 — Classless

Category

Miscellaneous / Reverse Engineering

Problem Description

The challenge provides a custom virtual machine binary (`objectvm`) and several `.bbl` (Babel) script files. The VM executes object-oriented programs encoded in a JSON-based bytecode that supports three language "dialects": `CPP`, `JAVA`, and `PY`. Each `.bbl` file encodes a compressed JSON object defining classes, vtables, objects, and a task.

The challenge name *Classless*, combined with the hint *"forget what a class is"*, implies that the exploit involves manipulating how the VM resolves class identity through vtables rather than through actual class definitions.

Note: An LLM was consulted to find the flag; however, the underlying concept was learned through that process.



Figure 10: Challenge files and initial analysis for Classless.

Approach and Solution

Step 1. Understand the `.bbl` file format: Each `.bbl` file is encoded as `zlib.compress(json.dumps` followed by Base64 encoding. Each payload contains:

- `classes[]` — class definitions with dialect, methods, slot numbers, and bodies.
- `objects[]` — instances with `declared_class`, `runtime_class`, and fields.
- `vtable entry` — the object ID to execute.

Step 2. Identify the hidden Vault mechanism: Examining the provided `.bbl` samples, Sample 04 reveals a hidden *Vault* that is triggered when an object carries `"__class__": "TrustedPlugin"` without a `__task__` field. The Vault performs three sequential checks:

- **Verifier** — checks whether the declared class matches `TrustedPlugin`.
- **Resolver** — performs a class resolution check via string comparison.
- **Dispatcher** — verifies the vtable slot at index 7.

All three checks must pass to trigger the flag decryption routine.

Step 3. Understand the decryption mechanism: Passing all three checks triggers an XOR decryption loop using key `0x37` over an embedded encrypted blob, yielding the flag. The binary additionally obfuscates the string `TrustedPlugin` using XOR with key `0x5a`.

Step 4. Static analysis approach: The flag can be extracted directly via static analysis by XOR-ing the encrypted blob located in `.rodata` at offset `0x134a0` with the key `0x37`, bypassing the need to execute the VM entirely.

```python
with open('objectvm', 'rb') as f:
    f.seek(0x134a0)
    data = f.read(40)
flag = bytes([b ^ 0x37 for b in data])
# → b'v1t{trilingual_vtable_babel_6f01a2c9}\x00...'
```

Figure 11: Static analysis output confirming the extracted flag.

Flag: `v1t{trilingual_vtable_babel_6f01a2c9}`

Challenge 8 — Atari 2600

Category

Miscellaneous / Game Hacking

Problem Description

The challenge provides an Atari 2600 game ROM. Participants are required to find a flag hidden within the game.

Approach and Solution

Step 1. Open the ROM in Stella: The game ROM was loaded using **Stella**, an Atari 2600 emulator that includes a built-in debugger. The game was played and observed to understand the objective and movement mechanics.

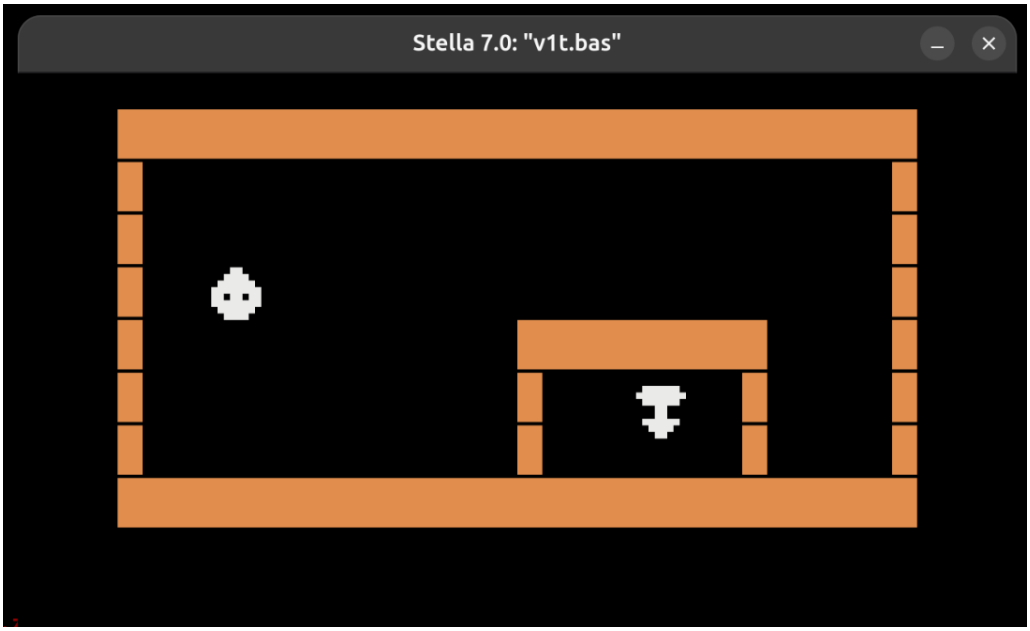


Figure 12: Game loaded in Stella, showing the initial state.

Step 2. Analyse via the Debugger: The Stella debugger was used to inspect RAM values and observe how the protagonist moves within the game frame.

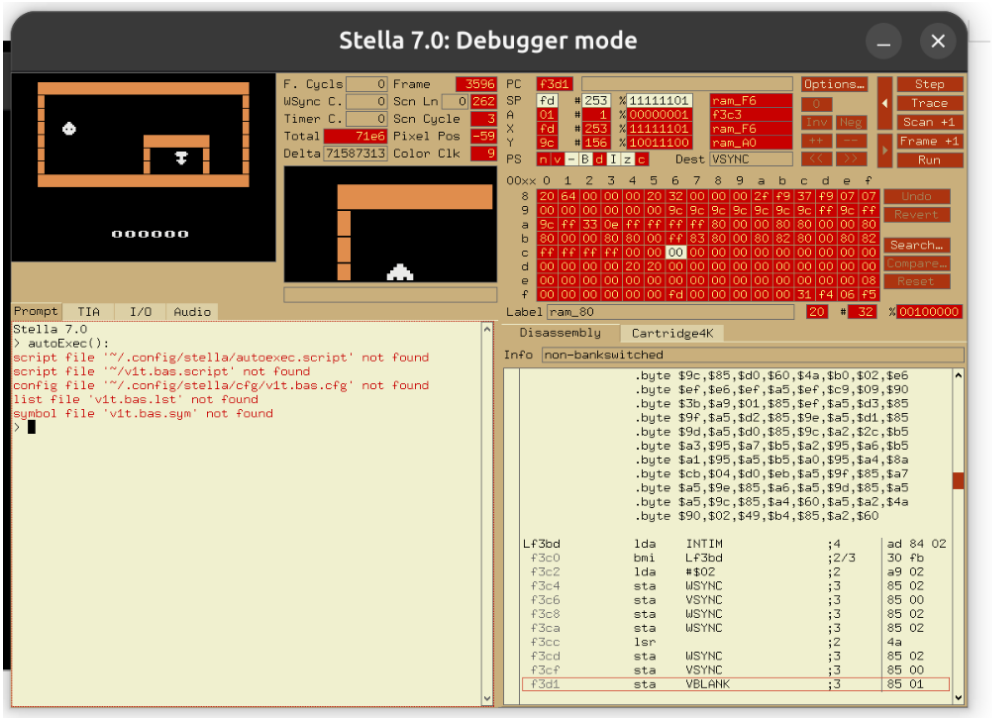


Figure 13: Stella debugger view showing RAM inspection.

Step 3. Manipulate RAM to remove walls: Specific RAM addresses controlling the wall positions were identified and modified, effectively removing the walls from the game. This allowed the protagonist to reach the target object freely.

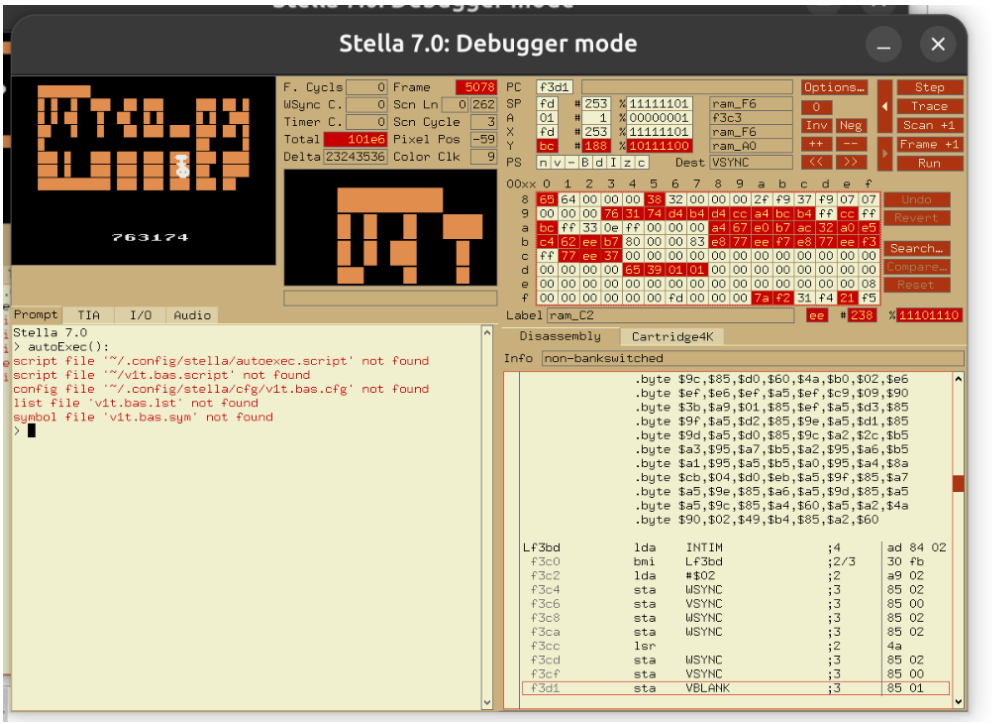


Figure 14: Game state after RAM manipulation, with walls removed.

Step 4. Retrieve the flag: After connecting the protagonist to the target from below,

the flag was displayed on screen.



Figure 15: Flag displayed upon successfully completing the game via RAM manipulation.

Flag: `v1t{0_0}`

Challenge 9 — Town Tour 03

Category

OSINT (Open Source Intelligence)

Problem Description

The challenge provides a photograph and asks participants to identify the exact location depicted. Using what3words.com, participants must determine the three-word address of the location to construct the flag.

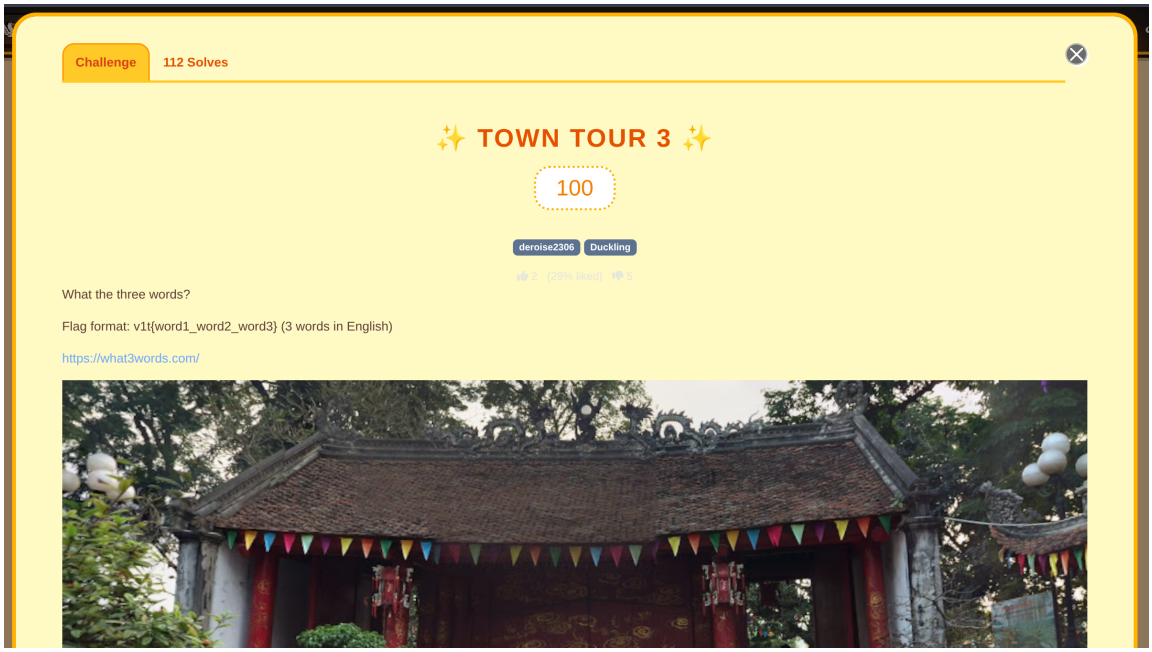


Figure 16: Challenge image provided for Town Tour 03.

Approach and Solution

Step 1. Identify the location: With the assistance of an LLM, the photograph was reverse-searched and the general location was identified.

Step 2. Obtain the what3words address: The identified coordinates were cross-referenced with what3words.com to obtain the three-word address.

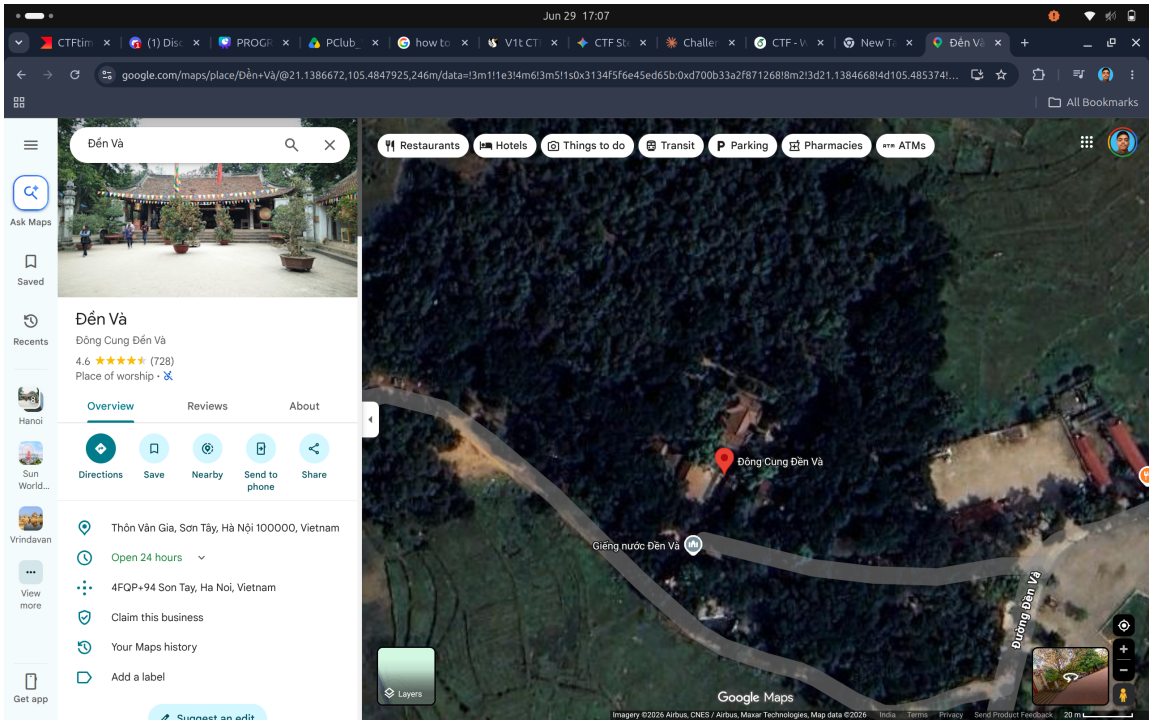


Figure 17: what3words lookup attempt for the identified location.

Step 3. Format the flag: The three-word address was retrieved; however, the flag could not be submitted correctly due to a formatting error.

Flag: *Not found — location identified but flag format could not be confirmed.*

End of Write-up